

1. A delegate is a type that can reference a method, while a lambda expression is a short way to write a method or function without giving it a name.
2. `var` is still statically typed because the compiler knows the variable's type at compile time based on the value assigned to it. The type is just not written explicitly.
3. An anonymous type is useful when we need to store a few related values temporarily, like getting a user's name and age from a database, without needing to create a whole class for them.



Search



Salma Mahmoud · You



Computer Science Student at October 6 Univer...

1h · 

Deferred Execution vs Eager Execution في C#

وأنا بتعلم LINQ، قابلتني فكرة بسيطة في شكلها، لكن مهمة جدًا في التعامل مع البيانات:

إمتى الـ Query بتننفذ فعلاً؟

◆ Deferred Execution

في الـ Deferred Execution، إحنا بنكتب الـ Query ونحدد إحنا عايزين إيه، لكن الـ Query نفسها مش بتننفذ في نفس اللحظة.

التنفيذ بيحصل لما نحتاج فعلاً للنتائج، زي لما نستخدم `foreach`.

```
var availableProducts = products
    .Where(p => p.Stock > 0);
```

هنا إحنا حددنا إننا عايزين المنتجات اللي عندها Stock، لكن الفترة نفسها لسه ما حصلتش.

لما نعمل:

```
foreach (var product in availableProducts)
{
    Console.WriteLine(product.Name);
}
```

هنا الـ Query بتننفذ.

◆ Eager / Immediate Execution

أحياناً بنحتاج إن الـ Query تنفذ فوراً ونحتفظ بالنتائج.

هنا ممكن نستخدم `methods` زي:

Tol ist()



Home



My Network



Post



Notifications



Jobs



Search



.ToList()
.ToArray()

مثال:

```
var availableProducts = products
    .Where(p => p.Stock > 0)
    .ToList();
```

هنا ال Query اتنفذت، والنتائج بقت موجودة في List.

Business Use Cases

تخيلي مثلاً إن عندنا E-commerce System.

مممكن يكون مفيد لما Deferred Execution:

* المنتجات والبيانات بتتغير باستمرار.
* عايزين ال Query تستخدم أحدث بيانات وقت التنفيذ.
* بنبي ال Query خطوة بخطوة قبل ما نطلب البيانات فعلاً.

مثال:

```
var products = db.Products
    .Where(p => p.IsActive);
if (category != null)
    products = products.Where(p => p.CategoryId ==
category);
```

هنا نقدر نكمل إضافة filters لل Query، وبعد كده ننفذها لما نحتاج النتائج.

أما Eager Execution ممكن يكون مفيد لما:

* نحتاج نسخة من النتائج في لحظة معينة.
* هنستخدم نفس النتائج أكثر من مرة.
* مش عايزين ال Query تتنفذ من جديد كل مرة نعمل فيها iteration.



Home



My Network



Post



Notifications



Jobs



Search



```
var products = db.Products
    .Where(p => p.IsActive)
    .ToList();
```

هنا النتائج اتخزنت في List، ونقدر نستخدمها بعد كده بدون إعادة تنفيذ نفس ال Query على المصدر.

الفكرة الأساسية ببساطة:

Deferred Execution → "هتنفذ لما تحتاجي"

Eager Execution → "هتنفذ دلوقتي وهديك النتائج"

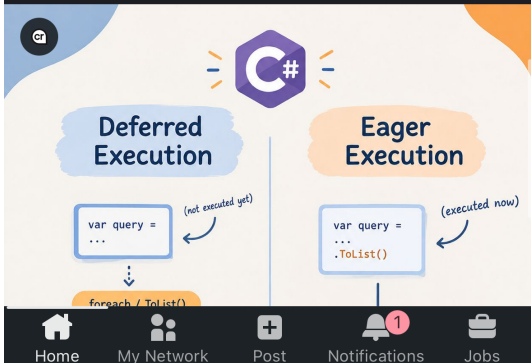
مفيش واحدة أفضل في كل الحالات.

المهم إننا نفهم إمتى ال data محتاجة تتجاب، وهل محتاجين Query بتنفذ وقت الاستخدام ولا محتاجين نحفظ بالنتائج نفسها.

وده كان من الحاجات اللي لفتت نظري وأنا بتعلم LINQ و .NET. 🚀

#CSharp #LINQ #DotNet #Programming
#SoftwareDevelopment #BackendDevelopment
#LearningInPublic #ComputerScience

Show translation





Search



مفيش واحدة أفضل في كل الحالات.

المهم إننا نفهم إمتي ال data محتاجة تتجيب، وهل محتاجين Query
تتنفذ وقت الاستخدام ولا محتاجين نحفظ بالنتائج نفسها.

وده كان من الحاجات الي لفتت نظري وأنا بتعلم LINQ و .NET. 🚀

#CSharp #LINQ #DotNet #Programming
#SoftwareDevelopment #BackendDevelopment
#LearningInPublic #ComputerScience

Show translation



Abdelrhman Fathy and 5 others



Home



My Network



Post



Notifications



Jobs